



UNIVERSITY *of* WEST FLORIDA

# Machine Learning with Scikit-learn

## Reinforcement Learning (RL)

Brian Jalaian, Ph.D.

*Associate Professor*

*Intelligent Systems & Robotics Department*

# Table of Content

- RL Overview  WE ARE HERE
- RL Foundations
  - Markov Decision Processes (MDPs)
  - Transition Dynamics and Environment Models
  - Episodic vs. Continuing Tasks
  - RL Terminology: Return, Policy, Value Function
  - Bellman Equations
- RL Algorithms
  - Dynamic Programming (DP)
  - Monte Carlo (MC) Methods
  - Temporal Difference (TD) Learning
  - TD – SARSA
  - TD – Q-learning

# RL Overview

# What is Reinforcement Learning?

We distinguish RL from supervised and unsupervised learning.

- **Supervised learning:** Learns from labeled data.
- **Unsupervised learning:** Finds structure in unlabeled data.
- **Reinforcement learning:** Learns through a sequence of interactions with an environment to maximize cumulative reward.
  - In RL, the "labels" (correct actions) are not predefined but learned through interactions to achieve a desired outcome.

| ML Paradigm                   | Key Feature                                  | Example                 |
|-------------------------------|----------------------------------------------|-------------------------|
| Supervised Learning           | Learns from labeled data                     | Predicting house prices |
| Unsupervised Learning         | Finds structure in unlabeled data            | Clustering customers    |
| <b>Reinforcement Learning</b> | Learns by <b>trial-and-error</b> interaction | Learning to play chess  |

# Core Ideas of RL

1. **No predefined labels:** Unlike supervised learning, RL doesn't know what the “correct” action is in advance.
2. **Delayed rewards:** The reward might not come immediately, e.g., in chess, the result is only known at the end of the game.
3. **Goal-oriented:** The agent's objective is defined through rewards for reaching **desired outcomes**, not via explicit instruction.

## Real-World Examples

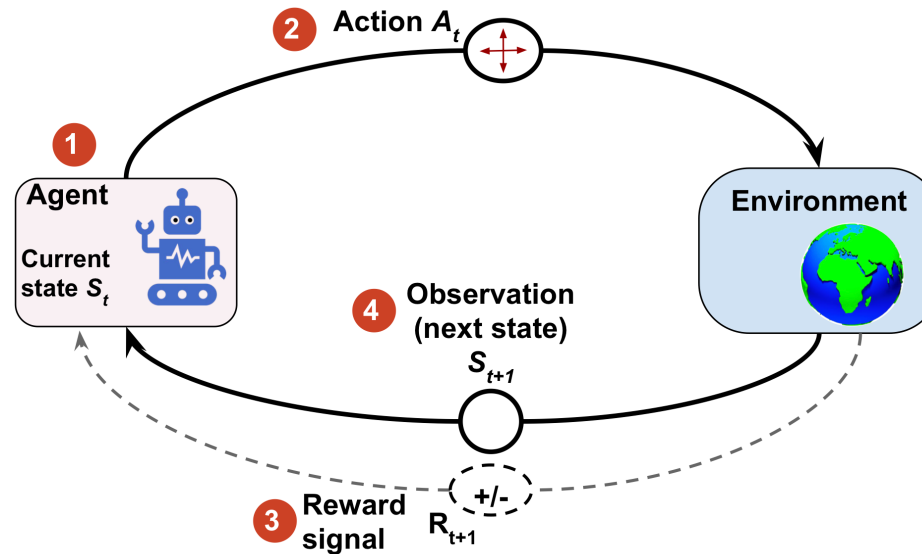
### 1. Playing Chess

- **Input:** Current configuration of the chess board.
- **Challenge:** Too many possible board configurations to label each as “good” or “bad”.
- **Solution:** Define the goal (winning), and reward only at the end.
  - If the agent wins, it receives a positive reward.
  - If the agent loses, it receives a negative reward.
- *“We don't tell the computer how to win. We only tell it what winning looks like.”*

### 2. Natural RL Analogy: Training a Dog

- The mechanism (e.g., how a seizure-alert dog detects seizures) may be unknown.
- Instead, behavior is shaped by rewarding successful outcomes (e.g., giving a treat for correctly alerting).

# Agent and Environment

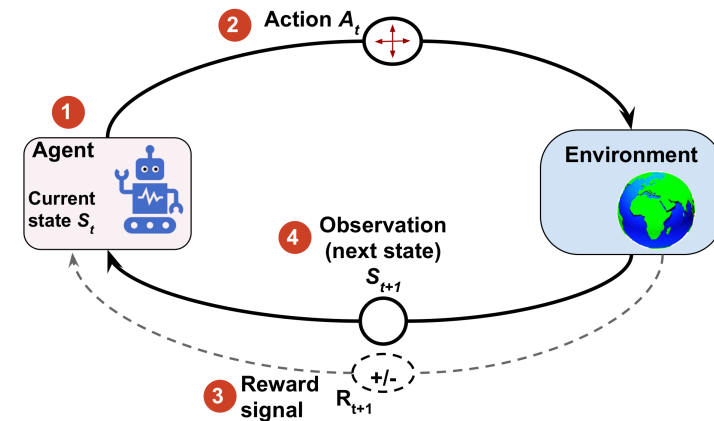


| Component   | Description                                                        |
|-------------|--------------------------------------------------------------------|
| Agent       | The learner/decision-maker; chooses actions based on state         |
| Environment | Everything external to the agent; returns observations and rewards |

# Agent-Environment Loop

At each time step:

1. The agent observes the current **state**  $S_t$ .
2. It chooses an **action**  $A_t$ .
3. The environment returns:
  - **Reward**  $R_{t+1}$
  - **Next state**  $S_{t+1}$



These interactions accumulate into an **episode** (a full sequence of interactions). Pay attention to the time step notations.

| Component      | Description                                                                                                   |
|----------------|---------------------------------------------------------------------------------------------------------------|
| <b>State</b>   | The set of variables representing the agent's current situation.                                              |
| <b>Action</b>  | The action taken by the agent in a given state.                                                               |
| <b>Reward</b>  | Feedback from the environment, indicating how well the agent performed (scalar value, +/-, natural language). |
| <b>Episode</b> | A sequence of agent-environment interactions.                                                                 |

# Exploration vs. Exploitation

## Exploration

- Trying **new or untested actions**.
- May yield better **long-term rewards**.
- Example: A student considering graduate school.

## Exploitation

- Choosing **known high-reward actions**.
- Focused on **immediate payoff**.
- Example: The same student deciding to take a job based on current qualifications.

**Trade-off:** Balancing these two is central to RL. There's no one-size-fits-all answer.

# Short Summary

## RL Challenges Highlighted

1. **Sequential dependency:** Future states depend on current actions.
2. **Delayed effects:** Rewards are not immediate; this complicates training.
3. **Non-i.i.d. data:** Experiences are correlated across time, violating i.i.d. assumptions used in supervised learning.
4. **Exploration vs. Exploitation:** The agent must decide when to try new actions vs. sticking with known good ones.
5. **Stability and convergence:** Learning can be unstable due to dynamic input distributions.

## Key Intuition

- RL is goal-oriented learning from experience.
- You don't tell the agent what to do—you reward outcomes.
- The agent figures out how to act via trial-and-error.

# Table of Content

- RL Overview
- RL Foundations  **WE ARE HERE**
  - Markov Decision Processes (MDPs)
  - Transition Dynamics and Environment Models
  - Episodic vs. Continuing Tasks
  - RL Terminology: Return, Policy, Value Function
  - Bellman Equations
- RL Algorithms
  - Dynamic Programming (DP)
  - Monte Carlo (MC) Methods
  - Temporal Difference (TD) Learning
  - TD – SARSA
  - TD – Q-learning

# RL Foundations

# What is an MDP?

An MDP provides a formal framework to model sequential decision-making problems under uncertainty.

An MDP is defined by:

- **A set of states  $\mathcal{S}$**
- **A set of actions  $\mathcal{A}$**
- **A transition probability function  $P(s', r \mid s, a)$**  = probability of transitioning to state  $s'$  and receiving reward  $r$  after taking action  $a$  in state  $s$
- **A reward function** (often embedded in the transition probability)
- **A discount factor  $\gamma \in [0,1]$**

# Agent–Environment Interaction in MDP

Interactions over time are represented as a sequence:

$$\{S_0, A_0, R_1\}, \{S_1, A_1, R_2\}, \{S_2, A_2, R_3\}, \dots$$

Where:

- $S_t$ : state at time  $t$
- $A_t$ : action at time  $t$
- $R_{t+1}$ : reward received after action  $A_t$

Note that  $S_t, R_{t+1}$ , and  $A_t$  are **time-dependent random variables** that take values from predefined finite sets denoted by  $s \in \hat{S}$ ,  $r \in \hat{R}$ , and  $a \in \hat{A}$ , respectively. In an MDP, these time-dependent random variables,  $S_t$  and  $R_{t+1}$ , have probability distributions that only depend on their values at the preceding time step,  $t - 1$ .

## Conditional Probability

The probability distribution for  $S_{t+1} = s'$  and  $R_{t+1} = r$  can be written as a conditional probability over the preceding state ( $S_t$ ) and taken action ( $A_t$ ) as follows:

$$p(s', r \mid s, a) \stackrel{\text{def}}{=} P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$$

This fully specifies the **environment dynamics**. If known, the MDP is **model-based**, because all transition probabilities of the environment can be computed. If unknown, we rely on **model-free** methods.

## Deterministic vs. Stochastic Environments

- **Deterministic:**  $p(s', r \mid s, a) \in \{0,1\}$ ; transitions are fixed and predictable - if you repeat  $(s, a)$ , you always get the same  $(s', r)$ .
  - If  $P(s', r \mid s, a) = 1$  for exactly one pair  $(s', r)$ , and all other pairs probabilities are 0, it's deterministic.
- **Stochastic:**  $p(s', r \mid s, a) \in [0,1]$ ; transitions have randomness.
  - If  $P(s', r \mid s, a) < 1$  for multiple  $(s', r)$ , it's stochastic.

## Marginalization

*What is the probability of ending up in state  $s'$ , given current state  $s$  and action  $a$ , regardless of reward?*

→ We only care about the probability of transitioning to the **next state**, regardless of the specific reward received. In that case, we **marginalize out** the reward by computing the **state-transition probability** (summing over rewards) in the stochastic environment:

$$p(s' | s, a) \stackrel{\text{def}}{=} P(S_{t+1} = s' | S_t = s, A_t = a)$$
$$p(s' | s, a) = \sum_r p(s', r | s, a)$$

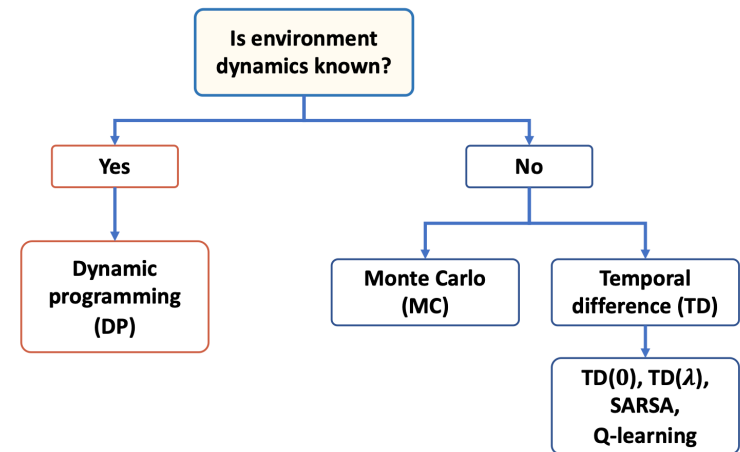
# Model-Based vs. Model-Free RL

**Model-based methods:** Dynamic Programming (DP)

**Model-free methods:**

- Monte Carlo (MC)
- Temporal Difference (TD)
- SARSA, Q-learning

These learn directly from **interaction samples** without estimating transition probabilities.



| Type        | Assumes known transition dynamics? | Requires sampling from environment? |
|-------------|------------------------------------|-------------------------------------|
| Model-based | Yes                                | Not necessarily                     |
| Model-free  | No                                 | Yes – relies on experience          |

# Episodic vs. Continuing Tasks

## Episodic Tasks:

- Interaction ends at a terminal state.
- Example: A chess game (starts and ends).
- Agent's experience is broken into episodes:

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_t, A_t, R_{t+1}, \dots, S_{t-1}, A_{t-1}, R_t, S_t$$

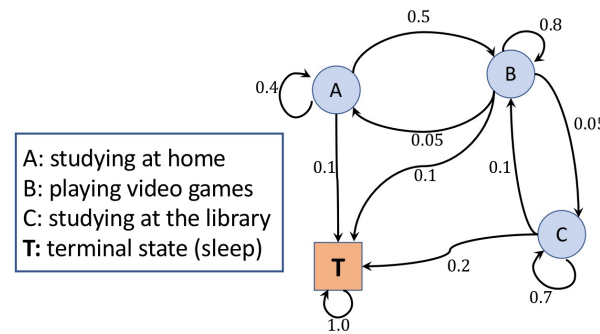
## Continuing Tasks:

- No terminal state; interaction goes on indefinitely.
- Example: A home-cleaning robot.

# Example of an MDP as a Markov Process

**Scenario:** A student chooses among 3 study situations every hour:

- **A:** Studying at home
- **B:** Playing video games
- **C:** Studying at the library
- **T:** Terminal state (sleep)



Transition probabilities:

|   | A    | B   | C    | T   |
|---|------|-----|------|-----|
| A | 0.4  | 0.5 | 0.0  | 0.1 |
| B | 0.05 | 0.8 | 0.05 | 0.1 |
| C | 0.0  | 0.1 | 0.7  | 0.2 |
| T | 0.0  | 0.0 | 0.0  | 1.0 |

**Transition probabilities** define likelihoods of moving from one state to another.

- Left: A **cyclic graph** with nodes (states) and edges (transitions).
- Right: A **transition table** showing probabilities from each state.

For example, assume that when staying at home (state A), there is a 50 percent likelihood that the student switches the activity to playing video games (state B).

All outgoing probabilities from a given state sum to 1.

We may formulate an episodic task like the following three examples:

- Episode 1: BBCCCCBAT  $\rightarrow$  pass (final reward = +1 )
- Episode 2: ABBBBBBBBBBBT  $\rightarrow$  fail (final reward = -1 )
- Episode 3: BCCCCCT  $\rightarrow$  pass (final reward = +1 )

# RL Terminology: Return, Policy, and Value Function



To define the key mathematical objects in reinforcement learning:

- **Return**  $G_t$  : how much reward to expect.
- **Policy**  $\pi(a \mid s)$  : how an agent behaves; rules for action-selection.
- **Value functions**  $v_\pi(s)$  : how good a state or action is. Expected return starting from state  $s$  under policy  $\pi$ .
- **Bellman equations**: recursive relationships that connect value functions to each other.

# RL Terminology: Return

## Definition:

The return  $G_t$  is the total reward the agent accumulates from time step  $t$  onward the entire duration of an episode.

## Finite Horizon / Episodic Tasks:

If the episode ends at time step  $T$ , then:  $G_t = R_{t+1} + R_{t+2} + \dots + R_T$

## Infinite Horizon / Continuing Tasks:

Use a discount factor  $\gamma \in [0,1]$  to prevent infinite accumulation:

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

This is a **geometric series** where:

- $\gamma = 0$  : agent cares only about **immediate reward  $r$**  (short-sighted agent).
- $\gamma \approx 1$  : agent cares about **long-term rewards**.

Express the return in a recursive way:

$$G_t = R_{t+1} + \gamma G_{t+1} = r + \gamma G_{t+1}$$

The book focuses on **episodic tasks** but uses discounting to generalize the return definition.

# RL Terminology: Return

**Previous example:** Assume  $\gamma = 0.9$  and that the only reward given is based on the result of the exam (+1 for passing the exam, and -1 for failing it). The rewards for intermediate time steps are 0.

Episode 1: BBCCCCBAT  $\rightarrow$  pass (final reward = +1 ): We have 8 steps in total ( $T = 8$ , index from  $t = 0$  to 7).

At  $t = 0$ , the model need to "predict" 7 steps for this episode.

$$t = 0: G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots + \gamma^6 R_7 = 0 + 0 \times \gamma + \cdots + 1 \times \gamma^6 = 0.9^6 \approx 0.531$$

At  $t = 1$ , the model need to "predict" 6 steps for this episode.

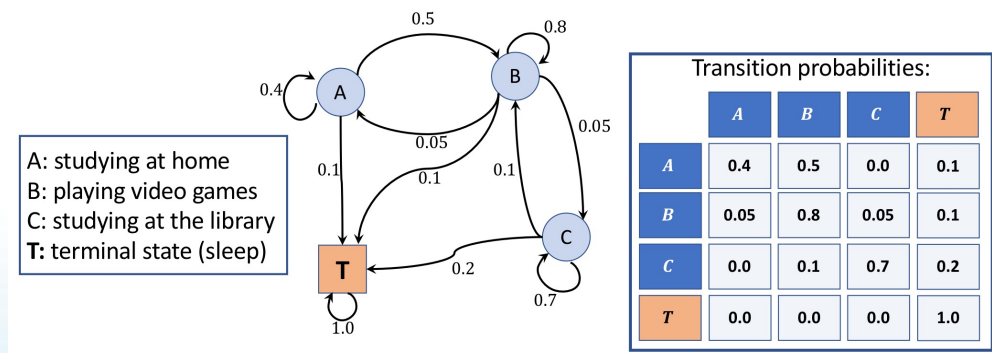
$$t = 1: G_1 = 1 \times \gamma^5 = 0.590$$

$$t = 2: G_2 = 1 \times \gamma^4 = 0.656$$

...

$$t = 6: G_6 = 1 \times \gamma = 0.9$$

$$t = 7: G_7 = 1 = 1$$



# RL Terminology: Return

Episode 2: *ABBBBBBBBBBT*  $\rightarrow$  fail (final reward =  $-1$ ):

$$t = 0: G_0 = -1 \times \gamma^8 = -0.430$$

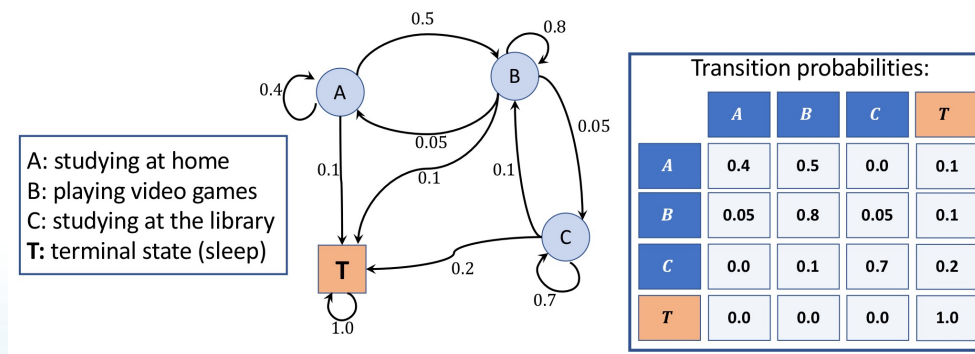
$$t = 1: G_0 = -1 \times \gamma^7 = -0.478$$

...

$$t = 8: G_0 = -1 \times \gamma = -0.9$$

$$t = 9: G_{10} = -1$$

Remember the reward is a signal that the agent receives when performing an action to transition from one state to the next. **Not every action yields a positive or negative reward.**



# RL Terminology: Policies

A **policy**  $\pi$  is a function that tells the agent what action to take in a given state.

**Stochastic Policy:** Gives the probability of choosing action  $a$  when in state  $s$ .

$$\pi(a | s) \stackrel{\text{def}}{=} P[A_t = a | S_t = s]$$

**Deterministic Policy:** Specifies exactly which action to take.

$$\pi(s) = a$$

RL agents aim to learn a policy that maximizes expected return.

# Value Functions

These describe how good a state or action is under a certain policy.

**State-Value Function:** Expected return (the average return over all possible episodes) starting in state  $s$ , following policy  $\pi$ .

$$v_{\pi}(s) \stackrel{\text{def}}{=} E_{\pi}[G_t \mid S_t = s] = E_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^{k+1} R_{t+k+1} \mid S_t = s \right]$$

**Action-Value Function:** Expected return from state  $s$ , taking action  $a$ , and following  $\pi$  afterward.

$$q_{\pi}(s, a) \stackrel{\text{def}}{=} E_{\pi}[G_t \mid S_t = s, A_t = a] = E_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^{k+1} R_{t+k+1} \mid S_t = s, A_t = a \right]$$

In practice, we store all the state values in a table denoted by  $V(s)$ . This could be a list or a NumPy array whose indices refer to different states; or, it could be a dictionary, where the dictionary keys map the states to the respective values.

# Bellman Equations

The **Bellman equation** expresses a value function in terms of **immediate reward** plus the **value of the next state**. The followings are recursive definitions of value functions.

## Bellman Expectation Equation for State-Value:

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} P(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

Interpretation:

- For each possible action  $a$ , weigh by  $\pi(a | s)$
- For each possible next state  $s'$  and reward  $r$ , compute:
  - Immediate reward  $r$
  - Discounted future value  $\gamma v_{\pi}(s')$
  - Take the expectation over all transitions

# Bellman Equations

Derivation:

Based on the recursive equation for the total return  $G_t = r + \gamma G_{t+1}$ , we can rewrite the value function as follows:

$$\begin{aligned} v_{\pi}(s) &\stackrel{\text{def}}{=} E_{\pi}[G_t \mid S_t = s] \\ &= E_{\pi}[r + \gamma G_{t+1} \mid S_t = s] \\ &= r + \gamma E_{\pi}[G_{t+1} \mid S_t = s] \end{aligned}$$

We can use the environment dynamics to compute the expectation by summing all the probabilities of the next state  $s'$  and the corresponding rewards  $r$  :

$$v_{\pi}(s) = \sum_{a \in \hat{A}} \pi(a \mid s) \sum_{s' \in \hat{S}, r \in R} p(s', r \mid s, a) [r + \gamma E_{\pi}[G_{t+1} \mid S_{t+1} = s']]$$

# Bellman Equations

## Bellman Expectation Equation for Action-Value:

Same process can be applied to action-value function, and we obtain:

$$q_{\pi}(s, a) = \sum_{s', r} P(s', r \mid s, a) \left[ r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right]$$

Interpretation:

- First step is deterministic: take action  $a$
- Next step includes all possible  $s', r$
- Follow policy  $\pi$  from state  $s'$  onward

Recall that  $S_t, R_{t+1}$ , and  $A_t$  are **time-dependent random variables** that take values from predefined finite sets denoted by  $s \in \hat{S}, r \in \hat{R}$ , and  $a \in \hat{A}$ , respectively.

# Summary Table of Key Functions

| Concept               | Mathematical Form   | Meaning                                                      |
|-----------------------|---------------------|--------------------------------------------------------------|
| Return                | $G_t$               | Total future reward                                          |
| Policy (stochastic)   | $\pi(a \mid s)$     | Probability of taking action in state                        |
| State-value function  | $v_\pi(s)$          | Expected return from state $s$ under policy $\pi$            |
| Action-value function | $q_\pi(s, a)$       | Expected return from $s$ , taking $a$ , then following $\pi$ |
| Bellman for $v_\pi$   | See the previous Eq | Defines $v_\pi(s)$ recursively                               |
| Bellman for $q_\pi$   | See the previous Eq | Defines $q_\pi(s, a)$ recursively                            |

# Table of Content

- RL Overview
- RL Foundations
  - Markov Decision Processes (MDPs)
  - Transition Dynamics and Environment Models
  - Episodic vs. Continuing Tasks
  - RL Terminology: Return, Policy, Value Function
  - Bellman Equations
- RL Algorithms  **WE ARE HERE**
  - Dynamic Programming (DP)
  - Monte Carlo (MC) Methods
  - Temporal Difference (TD) Learning
  - TD – SARSA
  - TD – Q-learning

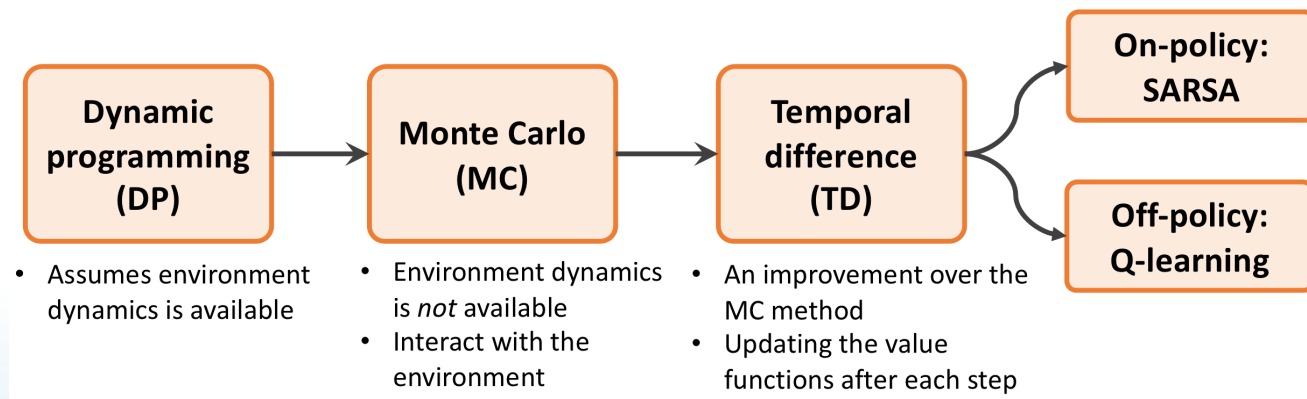
# RL Algorithms

# RL Algorithms

The three main classes of RL algorithms:

1. **Dynamic Programming (DP)**
2. **Monte Carlo (MC) Methods**
3. **Temporal Difference (TD) Learning**

Each class addresses the **core RL objective**: learning an optimal policy to maximize return. The methods differ in how they **estimate value functions**, how they **interact with the environment**, and whether they require knowledge of **transition dynamics**.



# Dynamic Programming (DP)

## Key Assumptions:

Full knowledge of environment dynamics (i.e., access to the MDP model). That is, all  $p(s', r \mid s, a)$  are known.

## Main Objectives:

1. Obtain the true state-value function,  $v_{\pi}(s)$ ; this task is also known as the prediction task and is accomplished with *policy evaluation*.
2. Find the optimal value function,  $v_{*}(s)$ , which is accomplished via *generalized policy iteration*.

# Dynamic Programming (DP)

## Limitations:

- Requires full model of the environment.
- Computationally expensive for large state spaces.

## Core DP Algorithms:

- **Policy Evaluation:** Predict  $v_{\pi}(s)$  for all  $s$ , given a policy  $\pi$ .
- **Policy Improvement:** Improve  $\pi$  by acting greedily w.r.t.  $v_{\pi}(s)$ .
- **Policy Iteration:** Alternate between evaluation and improvement until convergence.
- **Value Iteration:** Combine both steps into a single update rule using Bellman optimality.

# DP - Policy Evaluation

Goal: Estimate  $v_{\pi}(s)$  for all  $s$  under a fixed policy  $\pi$ .

## Iterative update:

We start from  $v^{(0)}(s)$ , which is initialized to zero values for each state. Then, at each iteration  $i + 1$ , we update the values for each state based on the Bellman equation, which, in turn, is based on the values of states from a previous iteration,  $i$ , as follows:

$$v^{\langle i+1 \rangle}(s) = \sum_a \pi(a | s) \sum_{s' \in \tilde{S}, r \in \hat{R}} p(s', r | s, a) [r + \gamma v^{\langle i \rangle}(s')]$$

It can be shown that as  $i \rightarrow \infty$ ,  $v^{(i)}(s)$  converges to the true state-value function,  $v_{\pi}(s)$ .

## DP - Policy Improvement

Once  $v_\pi(s)$  is known, a better policy  $\pi'$  can be expressed as:  $v_{\pi'}(s) \geq v_\pi(s) \forall s \in \hat{S}$

Improves current policy by choosing the action that yields the highest expected value.

## DP - Policy Iteration

Repeat until the policy stabilizes (converges).

- Policy evaluation (estimate  $v_\pi$  )
- Policy improvement (update  $\pi$  )

## DP - Value Iteration

Instead of full evaluation, combine policy evaluation and policy improvement into a single step can be more efficient:

$$v^{(i+1)}(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v(s')]$$

In this case, the updated value for  $v^{(i+1)}(s)$  is maximized by **choosing the best action out of all possible actions**, whereas in policy evaluation, the updated value was using the weighted sum over all actions.

# Monte Carlo (MC) Methods

**Key idea:** Estimate the value of a state  $s$  by averaging returns from episodes where  $s$  is visited.

Use **first-visit Monte Carlo prediction**:

- For each state  $s$ , consider only the **first time** it is visited in an episode
- Compute the return  $G_t$  from that point onward
- Average these returns across episodes

**Limitations:**

- Requires **complete episodes** to compute return.
- Only suitable for **episodic tasks**.

**Implementation:**

After generating a set of episodes, for each state,  $s$ , the set of episodes that all pass through state  $s$  is considered for calculating the value of state  $s$ .

- Maintain:
  - A count of how many times each state is first-visited
  - A running total of returns for each such state
- Update rule:  $v_{\pi}(s) \approx \frac{1}{N(s)} \sum_{i=1}^N G_t^{(i)}$

Where  $G_t^{(i)}$  is the return from episode  $i$ , starting at state  $s$ ;  $N(s)$  is the set of episodes.

# MC Control: Finding an Optimal Policy

**MC control** refers to the optimization procedure for improving a policy.

**Objective:** Use MC methods not just to evaluate a policy, but to optimize it - find  $\pi^*$

## Generalized Policy Iteration with MC:

1. Start with a random policy  $\pi_0$
2. Evaluate  $q_{\pi_0}$  using first-visit MC
3. Improve policy using greedy (deterministic) policy:  $\pi_1(s) \stackrel{\text{def}}{=} \arg \max_a q(s, a)$
4. Repeat until convergence:

$$\pi_0 \xrightarrow{\text{Eval.}} q_{\pi_0} \xrightarrow{\text{Improve}} \pi_1 \xrightarrow{\text{Eval.}} q_{\pi_1} \xrightarrow{\text{Improve}} \pi_2 \dots \xrightarrow{\text{Eval.}} q_* \xrightarrow{\text{Improve}} \pi_*$$

This is analogous to policy iteration in DP, but operates on **sampled episodes** rather than full MDP models.

# Temporal Difference (TD) Learning

## Hybrid Between DP and MC:

- **Model-free** like Monte Carlo
- **Bootstraps** like Dynamic Programming: updates based on **estimated value of next state**, not actual return
  - The term *bootstrapping* should not be confused with the bootstrap estimates used in ensemble learning.

## Advantages:

- Can be used in **continuing tasks** (not just episodic like MC)
- Does **not wait for full episodes** (unlike MC)
- Learns from **incomplete sequences**
- Can converge with less data

| Feature             | Dynamic Programming (DP) | Monte Carlo (MC)    | Temporal Difference (TD) |
|---------------------|--------------------------|---------------------|--------------------------|
| Requires Model      | Yes                      | No                  | No                       |
| Online Updates      | No (needs full sweep)    | No (end of episode) | Yes (per step)           |
| Works with episodes | Not required             | Yes                 | No                       |
| Bootstrapping       | Yes                      | No                  | Yes                      |
| Learning Target     | Bellman expectation      | Sample return       | One-step estimate        |

# TD Prediction

Update rule:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

Where:

- $V(S_t)$  : Current estimate of value at state  $S_t$
- $\alpha$  : Step-size (learning rate)
- $G_t$  : Target return
- $\gamma$  : Discount factor

# TD(0): One-step Temporal Difference

We can replace  $G_t$  to  $G_{t:t+1}$ , which indicates that the return obtained at time step  $t$  while considering 1 step after time step  $t$ .

## TD(0) Update Rule:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_{t:t+1} - V(S_t)]$$

Where:

- Target return:  $G_{t:t+1} \stackrel{\text{def}}{=} R_{t+1} + \gamma V(S_{t+1}) = r + \gamma V(S_{t+1})$ 
  - $R_{t+1}$  : Immediate reward
  - $\gamma$  : Discount factor
- Error (TD error):  $\delta_t = [G_{t:t+1} - V(S_t)]$

This is the difference between the **new information** and the **old estimate**. Recall that TD updates happen after every step, not after full episodes.

# TD( $\lambda$ ): $n$ -step Temporal Difference

Likewise, we can replace  $G_t$  to  $G_{t:t+n}$ , which indicates that the return obtained at time step  $t$  while considering  $n$  steps after time step  $t$ .

**TD ( $\lambda$ ) Update Rule:**

$$V(S_t) \leftarrow V(S_t) + \alpha[G_{t:t+n} - V(S_t)]$$

Where:

$$G_{t:t+n} \stackrel{\text{def}}{=} \begin{cases} R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) & \text{if } t+n < T \\ G_{t:T} & \text{otherwise} \end{cases}$$

# TD Control Problem

So far, we focused on: Estimating  $v_\pi(s)$  or  $q_\pi(s, a)$  under a fixed policy  $\pi$

Now we want to:

- Find the **optimal policy**  $\pi^*$  that maximizes expected return.
- This involves **learning and improving** policies simultaneously.

Next, to describe how **TD learning** can be used not just to evaluate a policy but to **optimize** it. The two foundational methods introduced here are:

- SARSA: On-policy TD control
- Q-learning: Off-policy TD control

Both methods learn action-value functions  $q(s, a)$ , and improve policies based on them. Mathematically, We want to learn:  $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$  From this, we can improve the policy:  $\pi(s) = \arg \max_a Q(s, a)$

A **generalized policy iteration (GPI)** approach involves alternating:

- **Policy evaluation:** Estimate  $Q(s, a)$
- **Policy improvement:** Update  $\pi(s)$  to be greedy (or  $\varepsilon$ -greedy) with respect to  $Q$

# SARSA – On-policy TD Control

**SARSA** refers to the quintuple  $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ . This algorithm learns the **Q-values** for the current policy and updates based on the **action actually taken**.

## Update Rule:

The on-policy TD control algorithm can be generalized to  $n$ -step TD, but here, let's take TD(0) as an example, where change state-value function to action-value function:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- $\alpha$  : learning rate
- $\gamma$  : discount factor

## Characteristics:

- **On-policy**: Learns the Q-values of the policy it is currently following (e.g.,  $\epsilon$ -greedy).
- Can learn from **exploration** actions.
- **Policy is improved gradually** using updated Q-values (e.g., by making the policy more greedy).

# Q-learning – Off-policy TD Control

The key difference from SARSA: the second action  $A_{t+1}$  is **not the actual action taken**, but the one with the **maximum estimated Q-value**.

## Update Rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

## Characteristics:

- **Off-policy:** Learns the optimal policy independently of the current behavior (policy); that's why it is called off-policy.
- Updates are based on the **greedy policy**, even if the agent is exploring. This makes Qlearning more aggressive in policy improvement.

# Q-learning – Off-policy TD Control

## Q-learning Algorithm Steps:

1. Initialize  $Q(s, a)$  arbitrarily.
2. For each episode:
  - Initialize state  $S$
  - Loop:
    - . Choose action  $A \sim \pi$  (e.g.,  $\epsilon$ -greedy based on  $Q$  )
    - . Take action  $A$ , observe reward  $R$  and next state  $S'$ ,
    - . Update:  $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
    - . Set  $S \leftarrow S'$
  - Repeat until  $S'$  is terminal

# Q-learning – Off-policy TD Control

## $\epsilon$ -greedy Policy:

- With probability  $\epsilon$ , take a random action.
- With probability  $1 - \epsilon$ , take the greedy action:  $a = \arg \max Q(s, a)$

$\epsilon$  may:

- Be constant (e.g., 0.1)
- Decay over time for more exploitation as learning progresses

# SARSA vs. Q-learning

## Intuition:

- **SARSA:** “This is what I did, so I’ll update based on it.”
- **Q-learning:** “This is what I should’ve done optimally, so I’ll update toward that.”

| Feature                    | SARSA (On-policy)                              | Q-learning (Off-policy)                 |
|----------------------------|------------------------------------------------|-----------------------------------------|
| Policy used in update      | Action actually taken                          | Greedy action (regardless of policy)    |
| Type                       | On-policy                                      | Off-policy                              |
| Sensitivity to exploration | More conservative, safer in noisy environments | More aggressive, optimistic updates     |
| Convergence                | To the policy being followed                   | To the optimal greedy policy            |
| Target                     | $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$         | $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$ |
| Real-world use             | More conservative learning                     | Faster convergence in many cases        |

*Let's go to the lecture code!*

# RL Code Implementation